

GOOGLE CAREER / INTERVIEW BLUEPRINT

DAY 2 — DETAILED EXECUTION PLAN & CHECKLIST

Friday, 11 September 2026

Day 2 Theme	From Python business logic → real API service
Total Target	2 hours 30 minutes
Primary Goal	Finish the Day 1 project carry-forward and expose the Python DocumentService through FastAPI
DSA	Two Pointers — 3 problems
Backend	Project skeleton + FastAPI + Pydantic + API tests
Python	No new standalone topic — integrate yesterday's Python work

Important: Day 2 is deliberately a carry-forward day. You completed DSA and Python on Day 1, but the Project 1 base skeleton + FastAPI setup is still pending. Therefore, the project work gets priority before adding new Python material.

1. DAY 2 OBJECTIVES

By the end of Day 2, you should have a working local backend service, not just an empty repository. Yesterday's pure-Python business logic should be reused rather than rewritten.

- ☐ Finish the Project 1 GitHub repository and backend directory structure.
- ☐ Create and activate a Python virtual environment.
- ☐ Install FastAPI, Uvicorn, and pytest.
- ☐ Build a minimal FastAPI application.
- ☐ Expose GET / and GET /health.
- ☐ Expose POST /documents using Pydantic request/response models.
- ☐ Connect the API layer to yesterday's DocumentService and DocumentRepository.
- ☐ Understand API layer → service layer → repository layer separation.
- ☐ Add API tests for success, validation, and not-found behavior.
- ☐ Complete three Two Pointer DSA problems and record Big-O and mistakes.
- ☐ Commit the work with meaningful Git commit messages and push to GitHub.

2. TIME-BLOCKED DAY 2 PLAN — 2 HOURS 30 MINUTES

Block	Time	Work	Outcome
1	30 min	Project 1 — base skeleton	Repository + folders + environment ready
2	60 min	FastAPI implementation	Working /, /health, /documents APIs
3	30 min	Python → API integration	FastAPI calls DocumentService → Repository
4	30 min	DSA — Two Pointers	3 problems + notes + Big-O

3. PROJECT 1 — FINISH THE BASE SKELETON (30 MIN)

Repository name: `ai-engineering-intelligence-platform`

Create the following structure:

```
ai-engineering-intelligence-platform/  
├── src/  
│   ├── main.py  
│   ├── api/  
│   │   └── __init__.py  
│   ├── services/  
│   │   └── __init__.py  
│   ├── models/  
│   │   └── __init__.py  
│   ├── repositories/  
│   │   └── __init__.py  
│   └── core/  
│       └── __init__.py  
├── tests/  
│   └── __init__.py  
├── docs/  
├── requirements.txt  
├── .gitignore  
└── README.md
```

Do not over-engineer this structure. The purpose is to establish a clean foundation that will grow later into a production-style AI engineering platform.

3.1 Python environment

```
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows CMD
.venv\Scripts\activate

pip install fastapi uvicorn pytest
pip freeze > requirements.txt
```

- ☐ Verify python --version.
- ☐ Verify pip --version.
- ☐ Verify pytest is available.
- ☐ Confirm .venv is not committed to Git.
- ☐ Create a sensible .gitignore for Python.

4. FASTAPI — BUILD THE FIRST REAL API (60 MIN)

Create `src/main.py`. Start with the smallest useful application. The goal is to understand how an HTTP request reaches Python code.

4.1 GET / — application endpoint

```
GET /

{
  "application": "AI Engineering Intelligence Platform",
  "status": "running",
  "version": "0.1.0"
}
```

4.2 GET /health — health endpoint

```
GET /health

{
  "status": "healthy"
}
```

4.3 POST /documents — first business endpoint

Request body:

```
{
  "name": "engine-requirement",
  "document_type": "requirement",
  "version": "1.0"
}
```

Expected response:

```
{
  "id": "doc-001",
  "name": "engine-requirement",
  "document_type": "requirement",
  "version": "1.0",
  "status": "created"
}
```

Run the service locally with:

```
uvicorn src.main:app --reload
```

Then verify the API through the browser, Swagger UI, or an HTTP client. FastAPI's interactive documentation should be available at `/docs`.

Do NOT add today: PDF processing, embeddings, vector databases, LLMs, RAG, agents, PostgreSQL, Redis, Docker, Kubernetes, GCP, React, or authentication.

5. INTRODUCE PYDANTIC REQUEST/RESPONSE MODELS

Do not treat raw dictionaries as your long-term API contract. Introduce explicit schemas.

Create a request model conceptually like:

```
DocumentCreate
- name
- document_type
- version
```

Create a response model conceptually like:

```
DocumentResponse
- id
- name
- document_type
- version
- status
```

Key concept to learn today:

API schema $\neq$ Database model $\neq$ Business object

- ☐ API schema defines what enters/leaves the HTTP boundary.
- ☐ Business object represents domain/business logic.
- ☐ Database model represents persistence.
- ☐ Keeping these concepts separate makes the system easier to evolve.

6. CONNECT YESTERDAY'S PYTHON CODE TO FASTAPI (30 MIN)

This is the most important architectural step of Day 2. Do not rewrite yesterday's Document, DocumentRepository, DocumentService, or custom exceptions just to make them fit FastAPI.

Target architecture:

```
HTTP Client
↓
FastAPI Router
↓
DocumentService
↓
DocumentRepository
↓
In-Memory Store
```

The request should flow like this:

```
POST /documents
↓
API Layer
↓
Service Layer
↓
Repository Layer
↓
Storage
```

The API layer should translate HTTP input into the business operation. The service layer should continue to own business rules. The repository should continue to own storage operations.

- ☐ Reuse yesterday's Document class/business model where appropriate.
- ☐ Reuse DocumentService.
- ☐ Reuse DocumentRepository.

- ☐ Reuse `DocumentNotFoundError` and `InvalidDocumentError`.
- ☐ Keep HTTP-specific behavior inside the API layer.
- ☐ Do not put business rules directly inside the FastAPI route handler.

7. FASTAPI ERROR HANDLING

Your Python exceptions need to become meaningful HTTP responses. This is the bridge between application engineering and API engineering.

- ☐ `DocumentNotFoundError` → HTTP 404.
- ☐ Invalid client input → an appropriate 4xx validation/client response.
- ☐ Unexpected application failure → do not silently return a fake success response.
- ☐ Understand the difference between a Python exception and an HTTP error response.

Mental model:

```

Python exception
  ↓
API error handling
  ↓
HTTP status code + response body
  
```

8. API TESTING

Add tests under `<b>tests/</b>`. You do not need a huge test suite today. You need a small suite proving that the API contract and architecture work.

- ☐ GET / → 200.
- ☐ GET /health → 200 and status is healthy.
- ☐ POST /documents → success response and document returned.
- ☐ Invalid request → validation error.
- ☐ Missing document → 404.
- ☐ Run pytest and confirm the tests pass.

The important engineering lesson is not the number of tests. It is that your API behavior is executable and repeatable rather than verified only by manually clicking around Swagger.

9. DSA DAY 2 — TWO POINTERS (30 MIN)

Today's pattern is `<b>Two Pointers</b>`. The goal is pattern recognition, not racing through many problems.

Problem	Primary idea
Valid Palindrome	Compare characters from both ends
Two Sum II	Move left/right pointers based on sorted sum
Best Time to Buy and Sell Stock	Track minimum seen so far and best profit

For each problem, record:

- ☐ Problem statement.
- ☐ Pattern used.
- ☐ Brute-force idea.
- ☐ Optimized approach.
- ☐ Time complexity.

- ☐ Space complexity.
- ☐ Why the optimized approach works.
- ☐ The mistake or observation you want to remember.

Recommended DSA structure after Day 2:

```
google-interview-prep/
├── dsa/
│   ├── arrays-hashmaps/
│   │   ├── two_sum.py
│   │   ├── contains_duplicate.py
│   │   └── valid_anagram.py
│   └── two-pointers/
│       ├── valid_palindrome.py
│       ├── two_sum_ii.py
│       └── best_time_to_buy_sell_stock.py
```

10. GIT DISCIPLINE — DAY 2 COMMITS

Use meaningful commits instead of one giant end-of-day commit. This will make the repository look like an engineering project rather than a dump of learning exercises.

- ☐ chore: initialize backend project structure
- ☐ feat: add FastAPI application and health endpoints
- ☐ feat: add document creation API
- ☐ refactor: connect document service to API layer
- ☐ test: add document API tests
- ☐ feat: add two pointer DSA solutions

Push both repositories to GitHub and verify that the README, source tree, tests, and commit history are visible.

11. END-OF-DAY REPOSITORY STATE

google-interview-prep

```
google-interview-prep/
├── dsa/
│   ├── arrays-hashmaps/
│   │   ├── two_sum.py
│   │   ├── contains_duplicate.py
│   │   └── valid_anagram.py
│   └── two-pointers/
│       ├── valid_palindrome.py
│       ├── two_sum_ii.py
│       └── best_time_to_buy_sell_stock.py
```

ai-engineering-intelligence-platform

```
ai-engineering-intelligence-platform/
├── src/
│   ├── main.py
│   ├── api/
│   ├── services/
│   ├── models/
│   ├── repositories/
│   └── core/
├── tests/
├── docs/
├── requirements.txt
├── .gitignore
└── README.md
```

12. MASTER DAY 2 CHECKLIST

Project Setup

- ☐ Create ai-engineering-intelligence-platform repository.
- ☐ Initialize Git.
- ☐ Create Python virtual environment.
- ☐ Create src, api, services, models, repositories, core, tests, and docs.
- ☐ Create .gitignore.
- ☐ Create requirements.txt.
- ☐ Create README.md.

FastAPI

- ☐ Install FastAPI, Uvicorn, and pytest.
- ☐ Create src/main.py.
- ☐ Start the FastAPI server.
- ☐ Verify GET /.
- ☐ Verify GET /health.
- ☐ Add POST /documents.
- ☐ Add Pydantic request model.
- ☐ Add Pydantic response model.
- ☐ Validate incoming data.
- ☐ Return a document response.

Architecture

- ☐ Understand API → Service → Repository.
- ☐ Reuse DocumentService.
- ☐ Reuse DocumentRepository.
- ☐ Reuse yesterday's custom exceptions.
- ☐ Keep business rules out of route handlers.
- ☐ Implement 404 handling.
- ☐ Implement validation/error handling.
- ☐ Keep in-memory storage for now.
- ☐ Do not add PostgreSQL yet.

Testing

- ☐ Test GET /.
- ☐ Test GET /health.
- ☐ Test POST /documents.
- ☐ Test invalid request.
- ☐ Test missing document.
- ☐ Run pytest.
- ☐ Confirm all tests pass.

DSA

- ☐ Understand the Two Pointer pattern.
- ☐ Solve Valid Palindrome.
- ☐ Solve Two Sum II.
- ☐ Solve Best Time to Buy and Sell Stock.
- ☐ Write Big-O for each.
- ☐ Record mistakes/observations.
- ☐ Commit DSA work.

GitHub

- ☐ Use meaningful commits.
- ☐ Push changes to GitHub.
- ☐ Verify repository structure.
- ☐ Verify README.
- ☐ Verify tests.
- ☐ Verify commit history.

13. WHAT NOT TO DO ON DAY 2

- ☐ PostgreSQL
- ☐ Redis
- ☐ Docker
- ☐ Kubernetes
- ☐ GCP
- ☐ LangChain
- ☐ LangGraph
- ☐ Vector databases
- ☐ Embeddings
- ☐ RAG
- ☐ Agents
- ☐ React
- ☐ Authentication
- ☐ PDF processing

These technologies are intentionally postponed. Adding them now would create breadth without giving you the engineering foundation needed for the later AI/ML system.

14. DAY 2 DEFINITION OF DONE

DAY 2

↓

DSA: 3 problems, Two Pointers, Big-O

BACKEND: FastAPI, Pydantic, API tests

↓

Document API → Service → Repository → In-memory storage

Success metrics:

- ☐ GET / returns 200.
- ☐ GET /health returns 200.
- ☐ POST /documents reaches DocumentService.
- ☐ DocumentService reaches DocumentRepository.
- ☐ A document is created in in-memory storage.
- ☐ Invalid input is rejected.
- ☐ Missing document maps to 404.
- ☐ pytest passes.
- ☐ Both repositories are pushed to GitHub.

15. STRATEGY ADJUSTMENT FROM DAY 1 → DAY 2

Your actual Day 1 completion:

- ☐ DSA — completed.
- ☐ Python — completed.
- ☐ Project base skeleton — not yet completed.
- ☐ FastAPI — not yet completed.

Therefore Day 2 is adjusted rather than treated as a failure. The plan deliberately carries the unfinished project work into Day 2.

Day 2 progression:

```

DSA
↓
Continue with Two Pointers

Python
↓
No new standalone topic

Project
↓
Finish Day 1 skeleton
↓
Build FastAPI
↓
Connect DocumentService
↓
Add API tests

```

After this foundation is complete, normal progression resumes from Day 3.

16. DAY 2 KEY ENGINEERING TAKEAWAYS

- ☐ FastAPI is the HTTP boundary; it should not become the place where all business logic lives.
- ☐ Pydantic models make API contracts explicit and validate incoming data.
- ☐ Services own business behavior and rules.
- ☐ Repositories isolate persistence/storage operations.
- ☐ In-memory storage is acceptable for the foundation phase.
- ☐ Exceptions in Python need deliberate translation into HTTP behavior.
- ☐ Tests make the API contract repeatable.
- ☐ A small, clean architecture is better than a large stack of premature technologies.

- The Python exercise from Day 1 is not separate work; it becomes the business-logic foundation of Project 1.